

Isomer Decoys via Count-Matched Shadow Sites

A bijection between real peptidoforms and decoy peptidoforms – worked examples and failure modes

Pioneer.jl – prepared for N. Wamsley

2026-07-03

1. The concept

Instead of moving *one* site of a real peptidoform to a nearby non-acceptor (“move-one-adjacent”, the current generator), build a **shadow set of decoy acceptor sites, count-matched per modification type**, and pair **each real peptidoform with one unique decoy peptidoform of the same composition** placed on shadow sites.

Rules (as specified):

1. For a peptide, for **each modification type**, pick as many **decoy sites** as there are real potential sites of that type, at **random** positions that are
 - (a) not a real site of that mod, and (b) not already a decoy site.
2. For **each true peptidoform** (all mods on real sites), generate **one** decoy peptidoform that (a) has the **same count of each mod type** and (b) uses **at least one decoy site**.
3. **No two true peptidoforms may map to the same decoy peptidoform** (unique decoys).

2. Worked setup: SGGSGGMG

Index: S1 G2 G3 S4 G5 G6 M7 G8.

- Real **phospho** sites: **S1, S4** (2 sites)
- Real **oxidation** site: **M7** (1 site)
- Draw **2 decoy phospho** sites + **1 decoy oxidation** site at random, avoiding real sites {1,4,7} and each other. Say: **decoy-phospho = {2, 3}**, **decoy-oxidation = {5}**.

Notation: p = phospho, o = oxidation, on the preceding residue.

3. The full bijection (all peptidoforms)

With max 2 phospho + max 1 ox, every real peptidoform pairs with exactly one decoy of the same composition, all decoy mods on shadow sites:

real peptidoform	composition	-> decoy peptidoform
Sp1 = phospho@1	1P	phospho@2 (.p2)
Sp4 = phospho@4	1P	phospho@3 (.p3)
phospho@{1,4}	2P	phospho@{2,3}

real peptidoform	composition	-> decoy peptidoform
ox@7	1O	ox@5
phospho@1 + ox@7	1P1O	phospho@2 + ox@5
phospho@4 + ox@7	1P1O	phospho@3 + ox@5
phospho@{1,4} + ox@7	2P1O	phospho@{2,3} + ox@5

Your worked cases, in sequence notation:

- SpGGSpGGMG (phospho@1,4) -> SGpGpSGGMG (phospho@2,3)
- SGGSpGGMoG (phospho@4, ox@7) -> SGpGSGoGMG or SGGpSGoGMG
- ...and if SGpGSGoGMG (phospho@2) is already used, then SpGGSGGMoG (phospho@1, ox@7) -> SGGpSGoGMG (phospho@3) – the other one.

4. Why it balances exactly (the bijection is not luck)

For a composition using **k** mods of a type, the number of **real** placements is $C(N_{\text{real}}, k)$ and the number of **decoy** placements is $C(N_{\text{decoy}}, k)$. Because we chose **$N_{\text{decoy}} = N_{\text{real}}$ per mod type**, these are **equal** for every **k** – so within each composition there is an exact **bijection** between real and decoy peptidoforms. Rule 3 (uniqueness) is therefore always satisfiable, and the null is **balanced 1:1** by construction (mirrors ID target:decoy). Example above: 1P has $C(2,1)=2$ reals and 2 decoys; 2P has $C(2,2)=1$ and 1; 1P1O has 2 and 2.

The assignment *within* a composition is arbitrary (any bijection); a stable choice is to sort both lists and pair by index, seeded for reproducibility.

5. Fragments: a multi-site permutation (still no Koina)

A decoy is derived from its paired real by permuting m/z, exactly as before but for **all moved sites at once**. For a fragment, the net shift is the change in the mass it *contains*:

```
mz_decoy = mz_real + (1/z) * sum_over_moved_mods[ mass_m * (contains(d_m) - contains(k_m)) ]
```

i.e. add a mod's mass if the fragment now contains the decoy site but not the real site, subtract if the reverse, per moved mod. Intensities/RT inherited from the paired real. Because a decoy moves *every* mod into the shadow region (not one), it is **not a near-twin** of any real – it shares fewer ions, so it is better separated in the deconvolution (less of the “tied at 0.5” behavior, less L2 dependence).

6. A denser example: 3 phospho sites

Peptide S T Y A G K (S1 T2 Y3 A4 G5 K6). Real phospho: {1,2,3}. Non-acceptor positions: {4,5,6}
-> decoy-phospho = {4,5,6} (only just enough). Bijection:

real	-> decoy
phospho@1 / @2 / @3 (3 mono)	phospho@4 / @5 / @6
phospho@{1,2} / {1,3} / {2,3} (3 di)	phospho@{4,5} / {4,6} / {5,6}
phospho@{1,2,3} (1 tri)	phospho@{4,5,6}

$C(3,k) = C(3,k)$ for all k – again exact.

7. Potential problems (the point of this note)

1. **Not enough non-acceptor positions.** An acceptor-rich peptide (e.g. STYSTYK: 6 S/T/Y, one K) has fewer non-acceptor positions than needed decoy sites $\rightarrow$ the shadow set can't be built. Needs a fallback (reuse positions across compositions? allow decoy sites that are *other-mod* acceptors? shrink the null for that peptide? – each has consequences for balance).
2. **Multi-mod position contention.** Decoy sites for *all* mod types must be mutually distinct and avoid *all* real sites. With several mods the free positions run out faster than any single mod suggests.
3. **Full displacement can be too easy for multi-site.** A real false localization on a di-phospho is usually **one** site wrong; a full-shadow decoy has **both** wrong $\rightarrow$ more site-determining ions disagree $\rightarrow$ easier to reject $\rightarrow$ **under-** estimates FLR for multi-site forms. (The “at least one decoy site” clause hints at a fix – decoys that keep some real sites and move others – but that breaks the clean $C(N,k)$ bijection and needs its own uniqueness accounting.)
4. **Per-peptide correlated difficulty.** The shadow region is drawn **once per peptide**, so *all* of that peptide's decoys share it. If the draw lands far from the real sites, every decoy for that peptide is easy (and vice versa) – difficulty is correlated within a peptide, adding variance to the FLR estimate. Drawing per-peptidiform decorrelates but costs the clean bijection.
5. **Bijection needs identical var-mod limits.** $N_{\text{decoy}} = N_{\text{real}}$ gives the exact match only if `max_var_mods` (and any motif/min limits) apply the same way to decoy sites. A decoy site inside a motif the real mod requires (e.g. phospho only in an S-P motif) would not be a legal *real* placement, so the counts can desync.
6. **Intensity approximation is looser.** Decoy fragments borrow the paired real's Prosit intensities; for a **fully-displaced** decoy that pattern is a worse proxy than for a one-site move (the real never had mods in the shadow region).
7. **Decoys still pass ID FDR.** Like any `is_decoy=false` localization decoy, a shadow decoy is a real sequence at real m/z $\rightarrow$ it passes ID FDR (~half the passing list, as we measured). Intended, but it inflates the raw ID count and must be excluded from ID reporting.
8. **Which real do we permute from?** The bijection pairs a decoy with **one** real peptidiform; we permute *that* real's fragments. Fine – but the pairing choice (within a composition) slightly changes each decoy's fragments, so the seed must be recorded for reproducibility.

8. Why it is still better than move-one-adjacent

- **Balanced 1:1 null** by construction (the bijection), vs the current ad-hoc “one decoy per real if a site exists.”
- **Not a near-twin** $\rightarrow$ avoids the 50/50 tie pileup that made adjacent decoys sit at fraction ~0.5 and **under-**populate the confident set.
- **Random shadow positions** $\rightarrow$ decoy-to-real distances ~ match the acceptor spacing distribution (the uniform argument), so the difficulty mix is representative – *except* for the multi-site full-displacement caveat (#3).
- **Cleaner multi-mod handling** (per-type shadow sets).

Open questions to decide before building: - Full-displacement vs “move a subset” (partial decoys) for multi-site forms – which better matches real errors (#3)? - Shadow sites **per-peptide** (correlated, clean bijection) vs **per-peptidiform** (decorrelated, messier) (#4)? - Fallback when non-acceptor positions are scarce (#1, #2)? - Validate against the standards' known FLR – as with every variant,

the decoy design that best reproduces the ground-truth FLR wins.